

Assembly as you can see from the program can become quite cumbersome to manage as the complexity of the project increases. One of the important feature of a programming language is that it should be readable, a long assembly program tends to be so complex that it can be difficult even for the person who coded the whole thing in first place to understand the code.

C

C is perhaps the most popular language for programming embedded systems. The reason for this being the fact that C can be considered to a middle-level language. It is much more readable than vanilla assembly and provides developers with powerful features which can make the programming easier. At the same time, C is known to provide a low-level access to the hardware which makes it a favorite among the programmers as it brings together best of both worlds into one package. Another main reason for C compilers being so famous is that the C compilers are available for a wide variety of platforms. These compilers have seen a major boost in performance in recent years, and depending on whom you ask, C nowadays is considered to be as fast as assembly. Portability is also a factor here as there are a host of cross compilers out there which take a source file in C and can create the executable for any platform and architecture.

Consider this code snippet for example, which resets bits for Port H on a Motorola 68HC12

```
PORTH &= 0xF5; // Changes bits 1 and 3 to zeros using C
PORTH &= ~0x0A; // Same as above but using inverting the bit mask
// which makes it easier to see which bits are
// cleared
```

This would translate to something like this in assembly:

```
BCLR PORTH,$0A ;Changes bits 1 and 3 to zeros for Motorola 68HC12
```

You can see how easy it is to manipulate the low level hardware from directly inside C. This makes it the go to language for most platforms out there. There are a couple of features that we'd like to highlight.

Bitfields

Bitfields allow the programmers to access memory in unaligned sections, and even in sections smaller than a byte. In C, a bit field is created using a struct.